

Module 1

Topics Covered

Chapter 1 — An Overview of C

- A Brief History of C
- C Is a Middle-Level Language
- C Is a Structured Language
- C Is a Programmer's Language
- Compilers vs. Interpreters
- The Form of a C Program
- The Library and Linking
- Separate Compilation
- Compiling a C Program
- C's Memory Map

Chapter 2 — Expressions

- The Basic Data Types
- Modifying the Basic Types
- Identifier Names
- Variables
- The Four C Scopes
- Type Qualifiers
- Storage Class Specifiers
- Variable Initializations
- Constants
- Operators
- Expressions

CHAPTER 1 — AN OVERVIEW OF C

A Brief History of C

C was invented and first implemented by Dennis Ritchie on a DEC PDP-11 running the Unix operating system. It evolved from a lineage of older languages:

- BCPL — developed by Martin Richards
- B — invented by Ken Thompson, influenced by BCPL
- C — developed in the 1970s from B

For many years, the de facto C standard was the version supplied with Unix, first described in *The C Programming Language* by Brian Kernighan and Dennis Ritchie (Prentice-Hall, 1978).

In summer 1983, a committee was formed to create an ANSI (American National Standards Institute) standard.

The standardization process took six years.

The ANSI C standard was finally adopted in December 1989, with the first copies available in early 1990. It was also adopted by ISO — together known as ANSI/ISO Standard C.

In 1995, Amendment 1 to the C standard added several new library functions.

The 1989 standard plus Amendment 1 became a base document for Standard C++.

This version is commonly referred to as C89.

During the 1990s, development of the C++ standard consumed most programmers' attention, but work on C continued.

The result was the C99 standard. C99 retained nearly all features of C89 and focused on:

- Addition of several numeric libraries
- New special-use features: variable-length arrays, restrict pointer qualifier

C Is a Middle-Level Language

C is often called a middle-level computer language.

This does not mean it is less powerful or less developed.

Rather, it combines the best elements of high-level languages with the control and flexibility of assembly language.

C's Place in the World of Programming Languages:

Level	Languages
High Level	Ada, Modula-2, Pascal, COBOL, FORTRAN, BASIC
Middle Level	Java, C++, C, FORTH, Macro-assembler
Low Level	Assembler

Key characteristics of C as a middle-level language:

- Allows manipulation of bits, bytes, and addresses — the basic elements with which the computer functions

- C code is highly portable — it is easy to adapt software written for one type of computer or OS to another
- Supports several built-in data types but is not strongly typed (unlike Pascal and Ada)
- Permits almost all type conversions — e.g., freely intermixing char and int in expressions
- Does not perform strict run-time error checking (e.g., no automatic array-boundary checks)
- Does not demand strict type compatibility between a parameter and an argument
- Allows direct manipulation of bits, bytes, words, and pointers — ideal for system-level programming
- Has a small number of keywords: C89 defines 32; C99 adds only 5 more (compared to BASIC's 100+)

C Is a Structured Language

C is commonly referred to as a structured language, sharing similarities with ALGOL, Pascal, and Modula-2.

The distinguishing feature of a structured language is compartmentalization of code and data — the ability to section off and hide information and instructions necessary to perform a specific task.

Structured vs. Non-Structured Languages:

Non-Structured	Structured
FORTRAN	Pascal
BASIC	Ada
COBOL	C++, C, Java, Modula-2

Key features of structured C programs:

- Functions — C's stand-alone subroutines; the main building blocks where all program activity occurs
- Local variables — used inside subroutines; allow events within them to cause no side effects elsewhere
- Code blocks — logically connected group of statements between { } treated as a unit
- Supports multiple loop constructs: while, do-while, for
- Use of goto is discouraged; not the common form of program control
- Allows statements to be placed anywhere on a line; does not require a strict field concept

C Is a Programmer's Language

C was created, influenced, and field-tested by working programmers.

It gives programmers what they want: few restrictions, few complaints, block structure, stand-alone functions, and a compact set of keywords.

C vs. Non-Programmer Languages:

- COBOL — designed for nonprogrammers to read programs
- BASIC — designed for nonprogrammers to solve simple problems
- C — created by and for programmers, offering near-assembly efficiency with structured-language organization

C was originally used for systems programming (operating systems, editors, compilers, linkers). Later it was adopted for all programming tasks because of its portability and efficiency.

C vs. Assembly Language:

- Assembly language maps one operation per CPU task — maximum flexibility but difficult to develop and debug
- Assembly-language programs are non-portable across CPUs
- C achieves near-assembly efficiency with the structure of Pascal or Modula-2

Despite predictions that C++ would replace C, C remains relevant because:

- Not all programs require object-oriented features (e.g., embedded systems)
- Much of the world runs on existing C code that continues to be maintained
- C99 shows that C is still a venue for leading-edge innovation

Compilers vs. Interpreters

A computer language defines the nature of a program, not the way it is executed. Two general execution methods exist:

Feature	Interpreter	Compiler
Operation	Reads source code one line at a time	Reads entire program at once
Output	Executes instructions directly (or via intermediary)	Produces object/binary/machine code
Speed	Slower — overhead on each run	Faster — compilation is a one-time cost
Runtime need	Interpreter must be present to run	Executable runs directly
Example	Earlier BASIC; Java (intermediary form)	C, C++

C was specifically designed and optimized as a compiled language. Although C interpreters exist (including as debugging aids), you will almost always use a C compiler when developing C programs.

The Form of a C Program

The 32 keywords defined by C89 (which also form the C subset of C++) are listed below:

C89 Keywords (32 total)
auto break case char const continue default do
double else enum extern float for goto if
int long register return short signed sizeof static
struct switch typedef union unsigned void volatile while

C99 adds five new keywords:

C99 Additional Keywords (5)

<code>_Bool</code> <code>_Complex</code> <code>_Imaginary</code> <code>inline</code> <code>restrict</code>
--

Additional rules for C programs:

- C is case-sensitive: `else` is a keyword, `ELSE` is not
- Keywords may not be used as variable or function names
- All C programs consist of one or more functions
- The function `main()` is the first function called when execution begins
- `main()` contains the outline of what the program does, composed of function calls
- Many compilers also support non-standard extended keywords (e.g., `asm`, `cdecl`, `far`, `huge`, `near`, `pascal`)

General form of a C program:

```
Global declarations

int main(parameter list)
{
    statement sequence
}

return-type f1(parameter list)
{
    statement sequence
}

return-type f2(parameter list)
{
    statement sequence
}
.
.
return-type fN(parameter list)
{
    statement sequence
}
```

The Library and Linking

Most C programs include calls to functions in C's standard library because C keywords alone do not provide input/output (I/O) operations, high-level mathematical computations, or character handling.

Important concepts:

- **Standard Library** — all C compilers include a standard library of functions for commonly needed tasks (I/O, math, string handling, etc.)
- **Standard C** specifies a minimal set of functions that all compilers must support
- Compilers may include additional functions beyond the standard (e.g., graphics functions)
- **Linking** — when you call a library function, the linker combines your compiled code with the object code from the standard library
- **Relocatable format** — library functions use offset-based memory addresses that are resolved into actual addresses during linking

- You can also put your own reusable functions into a library

Separate Compilation

C allows a program to be spread across two or more files, with each file compiled separately. Once compiled, all files are linked together with any library routines to form the complete object code.

Advantages of separate compilation:

- If one file is changed, only that file needs to be recompiled — saves substantial time on large projects
- Allows multiple programmers to more easily work together on a single project
- Provides a means of organizing code for large projects

Compiling a C Program

Creating an executable C program consists of three steps:

- Step 1: Creating your program (using an editor or integrated IDE)
- Step 2: Compiling your program
- Step 3: Linking your program with needed library functions

Important considerations:

- Most modern compilers supply integrated programming environments (IDE) with a built-in editor
- Stand-alone compilers require a separate editor
- Compilers accept only standard text files — files from word processors with control codes will not be accepted
- The exact compilation and linking method depends on the specific compiler and environment

C's Memory Map

A compiled C program creates and uses four logically distinct regions of memory:

Memory Region	Description
Program Code	Holds the program's executable code (machine instructions)
Global Variables	Stores all global variables; allocated in a fixed region by the compiler
Stack	Holds return addresses of function calls, function arguments, local variables, and current CPU state; dynamic and changing
Heap	A region of free memory your program can use via C's dynamic memory allocation functions (malloc, calloc, etc.)

The exact physical layout differs among CPU types and C implementations, but conceptually these four regions exist in every compiled C program.

CHAPTER 2 — EXPRESSIONS

The Basic Data Types

C89 defines five foundational data types:

Keyword	Data Type
char	Character
int	Integer
float	Floating-point
double	Double floating-point
void	Valueless

NOTE C99 adds three more data types: `_Bool`, `_Complex`, and `_Imaginary`. They are described in Part Two of the textbook.

Key points about basic data types:

- An object of type `char` is always 1 byte
- The size of `int` is usually the same as the word length of the execution environment (16-bit for DOS/Win 3.1; 32-bit for Windows 95/NT/2000)
- C stipulates only the minimal range of each data type, not its size in bytes
- The type `void` explicitly declares a function as returning no value, or creates generic pointers
- The range of `float` and `double` depends on the floating-point representation method; minimum range is $1\text{E}-37$ to $1\text{E}+37$

Modifying the Basic Types

Except for `void`, basic data types may be preceded by type modifiers that alter the meaning of the base type to more precisely fit a specific need. The modifiers are:

- `signed`
- `unsigned`
- `long`
- `short`

Valid modifier combinations:

- `int` — can be modified by `signed`, `short`, `long`, and `unsigned`
- `char` — can be modified by `unsigned` and `signed`
- `double` — `long` can be applied (long double)
- C99 also allows `long long` (for 64-bit integers)

Complete Data Types Table (Table 2-1):

Type	Typical Bits	Minimal Range
char	8	−127 to 127
unsigned char	8	0 to 255
signed char	8	−127 to 127
int	16 or 32	−32,767 to 32,767
unsigned int	16 or 32	0 to 65,535
signed int	16 or 32	Same as int
short int	16	−32,767 to 32,767
unsigned short int	16	0 to 65,535
signed short int	16	Same as short int
long int	32	−2,147,483,647 to 2,147,483,647
long long int (C99)	64	−(2 ⁶³ − 1) to 2 ⁶³ − 1
signed long int	32	Same as long int
unsigned long int	32	0 to 4,294,967,295
unsigned long long int (C99)	64	0 to 2 ⁶⁴ − 1
float	32	1E−37 to 1E+37, 6 digits precision
double	64	1E−37 to 1E+37, 10 digits precision
long double	80	1E−37 to 1E+37, 10 digits precision

Notes on signed/unsigned:

- signed on integers is allowed but redundant — the default is signed; its main use is to modify char in implementations where char is unsigned by default
- Signed integers: high-order bit is the sign flag (0 = positive, 1 = negative); negative numbers use two's complement
- Unsigned integers: high-order bit is part of the value, doubling the magnitude

Type modifier shorthand — when a type modifier is used alone, int is assumed:

Specifier	Same As
signed	signed int
unsigned	unsigned int
long	long int
short	short int

Identifier Names

In C, the names of variables, functions, labels, and other user-defined items are called identifiers.

Rules for identifiers:

- First character must be a letter (a–z, A–Z) or an underscore (_)
- Subsequent characters must be letters, digits (0–9), or underscores

- Upper- and lowercase are treated as distinct (count, Count, and COUNT are three separate identifiers)
- An identifier cannot be the same as a C keyword
- Identifiers should not have the same name as library functions

Examples:

Correct	Incorrect
count	1count (starts with digit)
test23	hi!there (contains !)
high_balance	high...balance (contains dots)

Identifier significance rules:

- C89: at least first 6 characters of external identifiers; at least first 31 of internal identifiers are significant
- C99: at least 31 significant characters for external; at least 63 for internal
- C++: at least first 1,024 characters are significant

Variables

A variable is a named location in memory that holds a value that can be modified by the program. All variables must be declared before use.

General declaration form:

```
type variable_list;
```

Examples:

```
int i, j, l;  
short int si;  
unsigned int ui;  
double balance, profit, loss;
```

Variables can be declared in three places:

- Inside functions — local variables
- In the definition of function parameters — formal parameters
- Outside of all functions — global variables

Local Variables

Variables declared inside a function (also called automatic variables) are local to that function.

- Known only within the block (between { }) in which they are declared
- Exist only while the block of code is executing — created upon entry, destroyed upon exit
- Cannot retain their values between calls (unless declared static)
- Stored on the stack by default
- A variable in an inner block with the same name as an outer block hides the outer variable

C89 rule: All local variables must be declared at the start of a block, before any action statements.

C99 rule: Local variables can be declared at any point within a block, prior to their first use.

Example — inner block hiding outer variable:

```
#include <stdio.h>
int main(void)
{
    int x;
    x = 10;
    if(x == 10) {
        int x; /* this x hides the outer x */
        x = 99;
        printf("Inner x: %d\n", x);
    }
    printf("Outer x: %d\n", x);
    return 0;
}
/* Output:
Inner x: 99
Outer x: 10 */
```

Formal Parameters

If a function uses arguments, it must declare variables to accept those values — called formal parameters.

- Behave like any other local variable inside the function
- Can be assigned values or used in any allowable expression
- Are local variables and are destroyed upon exit from the function

```
/* Return 1 if c is part of string s; 0 otherwise */
int is_in(char *s, char c)
{
    while(*s)
        if(*s == c) return 1;
        else s++;
    return 0;
}
```

Global Variables

Global variables are known throughout the program and may be used by any piece of code. They hold their value throughout the program's execution.

- Created by declaring them outside of any function
- Any expression may access them regardless of what block they are in
- Storage is in a fixed region of memory set aside by the compiler
- If a global and local variable share the same name, the local variable takes precedence within its block

Cautions with global variables:

- Take up memory the entire time the program executes
- Make a function less general because it relies on something defined outside itself

- Using too many globals can lead to program errors because of unknown and unwanted side effects

The Four C Scopes

Standard C defines four scopes that determine the visibility of an identifier:

Scope	Description
File scope	Starts at beginning of file; ends with end of file. Refers to identifiers declared outside all functions. Visible throughout the entire file. Variables with file scope are global.
Block scope	Begins with opening { of a block; ends with its closing }. Extends to function parameters. Variables with block scope are local to their block.
Function prototype scope	Identifiers declared in a function prototype; visible only within the prototype.
Function scope	Begins with opening { of a function; ends with closing }. Applies only to labels (targets of goto statements). A label must be within the same function as its goto.

Type Qualifiers

C89 defines two type qualifiers that control how variables may be accessed or modified: `const` and `volatile`. (C99 adds `restrict`.) Type qualifiers must precede the type names they qualify.

`const`

Variables of type `const` may not be changed by the program (though they can be given an initial value). The compiler is free to place them in read-only memory (ROM).

```
const int a = 10; /* a cannot be modified by the program */
```

Uses of `const`:

- Prevent modification of a variable within the program
- Prevent a function from modifying the object pointed to by a pointer argument
- Prove that any changes to a variable occur because of external events (e.g., hardware)

Example — `const` pointer parameter:

```
#include <stdio.h>
void sp_to_dash(const char *str);
int main(void)
{
    sp_to_dash("this is a test");
    return 0;
}
void sp_to_dash(const char *str)
{
    while(*str) {
        if(*str == ' ') printf("%c", '-');
    }
}
```

```
        else printf("%c", *str);
        str++;
    }
}
/* Output: this-is-a-test */
```

volatile

The volatile modifier tells the compiler that a variable's value may be changed in ways not explicitly specified by the program (e.g., by a hardware device or OS clock routine).

- Prevents the compiler from optimizing the variable by caching its value
- Prevents the compiler from reordering operations involving the variable

const and volatile can be used together:

```
const volatile char *port = (const volatile char *) 0x30;
/* Port value is set externally only, and the program cannot modify it */
```

Storage Class Specifiers

C supports four storage class specifiers that tell the compiler how to store a variable:

- extern
- static
- register
- auto

General declaration form:

```
storage_specifier type var_name;
```

NOTE Both C89 and C99 state that typedef is a storage class specifier for syntactic convenience, but it is not a storage class specifier in the common meaning of the term.

extern

C defines three categories of linkage:

- External linkage — functions and global variables available to all files in a program
- Internal linkage — file scope objects declared as static; known only within their file
- No linkage — local variables known only within their own block

The extern specifier is used to declare a variable without defining it — specifying that the object is defined with external linkage elsewhere in the program.

- Declaration — declares the name and type of an object
- Definition — causes storage to be allocated
- The same object may have many declarations, but only one definition
- Giving an extern variable an initialization turns the declaration into a definition

Most important use: in multiple-file programs, declare all global variables in one file and use extern declarations in the other files.

```

/* File One */
int x, y;
char ch;
int main(void)
{
    /* ... */
}

void func1(void)
{
    x = 123;
}

/* File Two */
extern int x, y;
extern char ch;
void func22(void)
{
    x = y / 10;
}

void func23(void)
{
    y = 10;
}

```

NOTE In real-world multiple-file programs, extern declarations are normally contained in a header file that is included with each source code file.

static Variables

Variables declared as static are permanent variables within their own function or file. Unlike global variables, they are not known outside their function or file, but they maintain their values between calls.

Static Local Variables:

- Compiler creates permanent storage for them (like a global variable)
- But they remain known only to the block in which they are declared
- A static local variable is a local variable that retains its value between function calls
- Very important for creating stand-alone functions that must preserve a value between calls

Example — static local variable in a series generator:

```

int series(void)
{
    static int series_num;
    series_num = series_num + 23;
    return series_num;
}

```

Static Global Variables:

- Applying static to a global variable restricts it to the file in which it is declared (internal linkage)
- Routines in other files have no knowledge of it and cannot alter its contents directly
- Useful when you want to hide portions of your program from other portions

register Variables

The register specifier requests that the compiler keep the value of a variable in a CPU register rather than in memory, for faster access.

- Originally applied only to int, char, or pointer types; C standard has since broadened it to any type
- Both C89 and C99 state: 'access to the object be as fast as possible'
- Can only be applied to local variables and formal parameters — not global variables

- You cannot obtain the address of a register variable using the & operator
- The compiler automatically converts register variables to normal variables if the limit is exceeded
- Most practical benefit is still with integer and character types

```
int int_pwr(register int m, register int e)
{
    register int temp;
    temp = 1;
    for(; e; e--) temp = temp * m;
    return temp;
}
```

auto

The auto keyword declares local (automatic) variables. Since all non-global variables are by default auto, this keyword is virtually never used and not commonly seen in modern C code.

Variable Initializations

Variables can be given a value at the time of declaration by placing an equal sign and a constant after the variable name.

General form:

```
type variable_name = constant;
```

Examples:

```
char ch = 'a';
int first = 0;
double balance = 123.23;
```

Initialization rules:

- Global and static local variables are initialized only once at program startup
- Local variables (excluding static) are initialized each time the block in which they are declared is entered
- Local variables that are not initialized have unknown (garbage) values before the first assignment
- Uninitialized global and static local variables are automatically set to zero

Constants

Constants refer to fixed values that the program may not alter. They can be of any basic data type. Constants are also called literals.

Character Constants

Enclosed between single quotes. For example, 'a' and '%' are character constants.

- C defines multibyte characters (one or more bytes) for large character sets — enclosed in single quotes, e.g., 'xy'
- Wide character constants (usually 16 bits) are preceded by L, e.g., L'A'; stored in wchar_t type

Integer and Floating-Point Constants

Integer constants are numbers without fractional components (10, −100). Floating-point constants require a decimal point (11.123) and may use scientific notation.

By default, the compiler fits a numeric constant into the smallest compatible data type. Exception: floating-point constants are assumed to be doubles.

Type Suffixes:

Data Type	Constant Examples
int	1 123 21000 −234
long int	35000L −34L
unsigned int	10000U 987u 40000U
float	123.23F 4.34e−3f
double	123.23 1.0 −0.9876324
long double	1001.2L
long long int (C99)	1234567890LL

Hexadecimal and Octal Constants

- Hexadecimal (base 16) — prefix 0x, uses digits 0–9 and A–F: e.g., 0x80 = 128 decimal
- Octal (base 8) — prefix 0, uses digits 0–7: e.g., 012 = 10 decimal

```
int hex = 0x80;    /* 128 in decimal */
int oct = 012;     /* 10 in decimal */
```

String Constants

A string is a set of characters enclosed in double quotes. For example, "this is a test". C allows string constants but does not formally have a string data type.

- Do not confuse strings with characters: 'a' is a character constant, "a" is a string containing one letter

Backslash Character Constants (Escape Sequences)

For characters that cannot be enclosed in single quotes (e.g., newline, carriage return), C provides backslash character constants:

Code	Meaning
\b	Backspace
\f	Form feed
\n	New line

Code	Meaning
\r	Carriage return
\t	Horizontal tab
\"	Double quote
\'	Single quote
\\	Backslash
\v	Vertical tab
\a	Alert (bell)
\?	Question mark
\N	Octal constant (N is octal)
\xN	Hexadecimal constant (N is hex)

Operators

C is very rich in built-in operators and places more significance on operators than most other languages. The four main classes are: arithmetic, relational, logical, and bitwise. There are also special-purpose operators.

The Assignment Operator

C uses a single equal sign (=) for assignment. Unlike most languages, the assignment operator can be used within any valid expression.

General form:

```
variable_name = expression;
```

Key terms:

- lvalue — an object that can appear on the left side of an assignment (a modifiable lvalue is a variable)
- rvalue — an expression on the right side of an assignment; represents the value of an expression

Type Conversion in Assignments: The value on the right side is converted to the type of the left side (target variable).

Target Type	Expression Type	Possible Info Loss
signed char	char	If value > 127, target is negative
char	short int	High-order 8 bits
char	int (16 bits)	High-order 8 bits
char	int (32 bits)	High-order 24 bits
char	long int	High-order 24 bits
short int	int (16 bits)	None

Target Type	Expression Type	Possible Info Loss
short int	int (32 bits)	High-order 16 bits
int (16 bits)	long int	High-order 16 bits
int (32 bits)	long int	None
long int (32 bits)	long long int (64 bits)	High-order 32 bits (C99 only)
int	float	Fractional part and possibly more
float	double	Precision, result rounded
double	long double	Precision, result rounded

Multiple Assignments — assign the same value to multiple variables in one statement:

```
x = y = z = 0;
```

Compound Assignments (Shorthand) — a more compact way to write variable = variable operator expression:

```
x += 10; /* same as x = x + 10 */
x -= 100; /* same as x = x - 100 */
x *= 2; /* same as x = x * 2 */
x /= 4; /* same as x = x / 4 */
x %= 3; /* same as x = x % 3 */
```

Arithmetic Operators

Operator	Action
–	Subtraction; also unary minus (negation)
+	Addition
*	Multiplication
/	Division (truncates remainder for integers)
%	Modulus — remainder of integer division (not for floats)
—	Decrement (subtract 1)
++	Increment (add 1)

Arithmetic operator precedence (highest to lowest):

```
Highest:  ++ --
           - (unary minus)
           * / %
Lowest:   + -
```

The Increment and Decrement Operators

The ++ and — operators add or subtract 1 from their operand. They may precede (prefix) or follow (postfix) the operand.

- Prefix (++x): increment/decrement is performed before the value is used in the expression
- Postfix (x++): the value is obtained first, then increment/decrement is performed

```
x = 10;
y = ++x; /* y = 11, x = 11 (prefix: increment first) */

x = 10;
y = x++; /* y = 10, x = 11 (postfix: use value first) */
```

Relational and Logical Operators

Relational operators compare values. Logical operators connect relational expressions. In C, true is any nonzero value; false is zero. Relational and logical expressions return 0 (false) or 1 (true).

Relational Operator	Action
>	Greater than
>=	Greater than or equal
<	Less than
<=	Less than or equal
==	Equal
!=	Not equal

Logical Operator	Action
&&	AND — true if both operands are true
	OR — true if either operand is true
!	NOT — reverses the logical state of its operand

Truth Table for Logical Operators:

p	q	p && q	p q	!p
0	0	0	0	1
0	1	0	1	1
1	1	1	1	0
1	0	0	1	0

Relative precedence of relational and logical operators (highest to lowest):

```
Highest:  !
          > >= < <=
          == !=
          &&
Lowest:   ||
```

NOTE All relational and logical operators are lower in precedence than arithmetic operators. For example, $10 > 1+12$ is evaluated as $10 > (1+12)$.

Bitwise Operators

C supports a full complement of bitwise operators for testing, setting, or shifting the actual bits in a byte or word. They apply to char and int types and their variants. They cannot be used on float, double, long double, or void.

Operator	Action
&	Bitwise AND
	Bitwise OR
^	Bitwise Exclusive OR (XOR)
~	One's Complement (NOT) — reverses all bits
>>	Shift right
<<	Shift left

XOR Truth Table:

p	q	p ^ q
0	0	0
1	0	1
1	1	0
0	1	1

Common uses of bitwise operators:

- Bitwise AND (&) — clear specific bits; mask off certain bits (e.g., parity bit in modem data)
- Bitwise OR (|) — set specific bits to 1
- Bitwise XOR (^) — toggle bits; true only when operand bits are different
- One's complement (~) — reverse all bits; used in cipher routines
- Shift operators (<<, >>) — multiply/divide by powers of 2

Shift Operator Effects (Table 2-7):

Statement	Binary Value	Decimal
x = 7;	00000111	7
x = x<<1;	00001110	14
x = x<<3;	01110000	112
x = x<<2;	11000000	192 (bits shifted off!)
x = x>>1;	01100000	96
x = x>>2;	00011000	24

NOTE Bitwise operations may produce any arbitrary value, whereas logical operators always evaluate to exactly 0 or 1. Shifted-off bits are permanently lost (not rotated).

The ? Operator (Ternary)

The ternary ? operator replaces certain if-then-else statements. It is C's most compact conditional expression.

General form:

```
Exp1 ? Exp2 : Exp3;
```

If Exp1 is true, Exp2 becomes the value; if Exp1 is false, Exp3 becomes the value.

```
x = 10;
y = x > 9 ? 100 : 200;    /* y = 100, because x > 9 is true */

/* Equivalent if-else: */
if (x > 9) y = 100;
else y = 200;
```

The & and * Pointer Operators

A pointer is the memory address of an object. C provides two operators for manipulating pointers:

Operator	Name	Action
&	Address-of	Unary operator that returns the memory address of its operand. Think of it as 'the address of'.
*	At-address (Dereference)	Unary operator that returns the value of the object at the address that follows it. Think of it as 'at address'.

```
int count = 100;
int *m;
int q;

m = &count; /* m receives the address of count (e.g., 2000) */
q = *m;     /* q receives the value at address m (= 100) */
```

Pointer variable declaration — put * in front of the variable name:

```
char *ch;          /* ch is a pointer to a character */
int x, *y, count; /* x and count are int; y is pointer to int */
```

- The type a pointer points to is called the base type
- Both & and * have higher precedence than all arithmetic operators except unary minus
- The multiplication symbol * and 'at address' * are the same character but have no relationship

The sizeof Operator

`sizeof` is a unary compile-time operator that returns the length in bytes of the variable or parenthesized type specifier that it precedes. It is evaluated at compile time, and the value is treated as a constant.

```
double f;
printf("%d ", sizeof f);          /* prints 8 (on most systems) */
printf("%d", sizeof(int));        /* prints 4 (on 32-bit system) */
```

- To compute the size of a type, enclose the type name in parentheses
- The return type is `size_t` (loosely, an unsigned integer)
- Primary use: generate portable code that depends on the size of built-in data types

The Comma Operator

The comma operator strings together several expressions. The left side is always evaluated as void; the value of the right side becomes the value of the entire comma-separated expression.

```
x = (y=3, y+1);
/* First assigns y=3, then assigns x the value y+1 = 4 */
```

The comma operator has the lowest precedence of all operators. Parentheses are necessary when using it on the right side of an assignment.

The Dot (.) and Arrow (→) Operators

These operators are used to access individual elements of structures and unions.

- `.` (dot) — used when working directly with a structure or union variable
- `→` (arrow) — used with a pointer to a structure or union

```
struct employee { char name[80]; int age; float wage; } emp;
struct employee *p = &emp;

emp.wage = 123.23; /* using dot operator directly */
p->wage = 123.23;  /* using arrow operator via pointer */
```

The [] and () Operators

- Parentheses () — operators that increase the precedence of operations inside them
- Square brackets [] — perform array indexing; expression inside provides an index into an array

```
char s[80];
s[3] = 'X';          /* assigns to element at index 3 */
printf("%c", s[3]); /* prints X */
```

Precedence Summary — All C Operators (Table 2-8)

Precedence	Operators
Highest	() [] → .
	! ~ ++ — -(type) * & sizeof

Precedence	Operators
	* / %
	+ -
	<< >>
	< <= > >=
	== !=
	&
	^
	&&
	?:
	= += -= *= /= etc.
Lowest	,

NOTE All operators except unary operators and ?: associate from left to right. Unary operators (*, &, -) and ?: associate from right to left.

Expressions

Operators, constants, functions, and variables are the constituents of expressions. An expression in C is any valid combination of these elements.

Order of Evaluation

C does not specify the order in which subexpressions of an expression are evaluated. This leaves the compiler free to rearrange an expression to produce more optimal code. Therefore, code should never rely upon the order in which subexpressions are evaluated.

```
x = f1() + f2(); /* does not ensure f1() is called before f2() */
```

Type Conversion in Expressions

When constants and variables of different types are mixed in an expression, all operands are converted to the type of the largest operand — called type promotion.

Type Conversion Algorithm (applied operation by operation after integral promotion):

First: All char and short int values are automatically elevated to int (integral promotion).

```
IF an operand is long double THEN second is converted to long double
ELSE IF an operand is double THEN second is converted to double
ELSE IF an operand is float THEN second is converted to float
ELSE IF an operand is unsigned long THEN second is converted to unsigned long
ELSE IF an operand is long THEN second is converted to long
```

```
ELSE IF an operand is unsigned int THEN second is converted to unsigned int
```

Special case: If one operand is long and other is unsigned int, and the unsigned int value cannot be represented by long, both are converted to unsigned long.

Casts

You can force an expression to be of a specific type using a cast.

General form:

```
(type) expression
```

```
(float) x / 2    /* force x/2 to be evaluated as float */

/* Example: print integer loop variable with fractional division */
for(i=1; i<=100; ++i)
    printf("%d / 2 is: %f\n", i, (float) i / 2);
```

- Casts are technically operators — unary, with the same precedence as other unary operators
- Without the cast, integer division would be performed and the fractional part lost

Spacing and Parentheses

Tabs and spaces may be freely added to expressions to improve readability — they do not affect execution.

Redundant or additional parentheses do not cause errors or slow execution. Use parentheses liberally to clarify the order of evaluation:

```
x = y/3-34*temp+127;    /* harder to read */
x = (y/3) - (34*temp) + 127; /* clearer with parentheses */
```

— End of Notes —